Module 1: Introduction to Node.js & Development Environment

1.0 Module Overview

Weightage: 4 Hours Lecture | 6 Hours Practical **Objective:** To move beyond the hype of Node.js and understand the architectural reasons for its existence. By the end of this module, you should not only have Node installed but understand *why* it behaves differently from traditional multi-threaded servers like Apache or Tomcat.

1.1 Introduction to Node.js

1.1.1 Definition

Official Definition: Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser.

The "Real" Definition: Node.js is a wrapper around Google Chrome's V8 JavaScript Engine. It takes the engine that makes Chrome fast, rips it out of the browser, and adds C++ bindings (via a library called `libuv`) to let it interact with the file system, network, and operating system. It allows you to write server-side code in the same language you use for client-side DOM manipulation.

1.1.2 Historical Context

- **Origin:** Created by Ryan Dahl in 2009.
- **The Problem:** Dahl was frustrated with the limited capabilities of the Apache HTTP Server to handle high concurrency. Traditional servers created a new thread for every incoming request, which consumed massive amounts of RAM and led to "thread blocking."
- **The Solution:** Dahl looked at JavaScript (specifically the V8 engine) because it was single-threaded and non-blocking by design. He combined it with an event loop to handle high concurrency with low resource usage.

1.2 Architecture: The Node.js Process Model vs. Traditional Models

This is the most critical theoretical concept in this module. If you get an exam question asking to "Compare Node.js with traditional servers," this is what they want.

1.2.1 Traditional Web Server Model (Multi-Threaded)

- **Examples:** Apache, Tomcat, older PHP setups.
- **Mechanism:** Request/Response Multi-threading.
- **Flow:**
    1. Client sends a request.
    2. Server assigns a dedicated **Thread** to handle that request.

3. If the request involves a Database query (I/O), the thread **waits** (blocks) until the data comes back.
4. Once data returns, the thread sends the response and becomes free.

- **Flaw:** If you have 8GB of RAM and each thread takes 2MB, you hit a hard limit on concurrent users. When threads run out, new users wait.

1.2.2 The Node.js Model (Single-Threaded Event Loop)

- **Mechanism:** Single-Threaded, Non-Blocking I/O.
- **Flow:**
  1. Node has **one** main thread for execution.
  2. Client sends a request.
  3. Node registers the request and an associated **callback function**.
  4. It offloads the heavy lifting (Database, File System) to the OS kernel (via `libuv`).
  5. The main thread **does not wait**. It immediately takes the next request.
  6. When the heavy task finishes, the callback is pushed to the **Event Queue**.
  7. The **Event Loop** picks it up and executes the callback.

1.2.3 Comparative Diagram (Conceptual)

Feature	Traditional (Apache/PHP)	Node.js
Architecture	Multi-threaded, Blocking I/O	Single-threaded, Non-blocking I/O
Concurrency	Limited by RAM/Threads	High concurrency (thousands of connections)
Context Switching	High (OS switches between threads)	Low (No thread switching overhead)
Best For	CPU Intensive tasks (Video encoding)	I/O Intensive tasks (Chat, Streaming, APIs)

1.3 Worked Examples: Blocking vs. Non-Blocking

To pass the exam, you must be able to write pseudocode demonstrating the difference.

Example 1: Blocking Code (Traditional Style)

Imagine a waiter who takes an order, walks to the kitchen, and stands there waiting for the chef to cook the food before serving it and taking the next table's order.

```
// Pseudo-code for Blocking
const fs = require('fs');
```

```
const data = fs.readFileSync('/file.md'); // Execution STOPS here
console.log(data); // This runs only after file is read
console.log("Next task..."); // This waits for the file read
```

Example 2: Non-Blocking Code (Node.js Style)

Imagine a waiter who takes an order, gives the ticket to the kitchen, and immediately goes to the next table. When the food is ready, a bell rings (event), and they serve it.

```
// Actual Node.js Code
const fs = require('fs');

fs.readFile('/file.md', (err, data) => {
    if (err) throw err;
    console.log(data); // Runs later, when file is ready
});

console.log("Next task..."); // Runs IMMEDIATELY, before file is read
```

1.4 Advantages, Use Cases, and Real-World Examples

1.4.1 Advantages

1. **High Performance:** V8 compiles JS directly to machine code.
2. **Scalability:** Handles thousands of concurrent connections with minimal hardware.
3. **Full Stack JS:** Developers use the same language for Backend and Frontend.
4. **NPM (Node Package Manager):** The largest ecosystem of open-source libraries in the world.

1.4.2 When to Use Node.js (Green Zone)

- **Real-time Applications:** Chat apps (WhatsApp clones), Collaboration tools (Trello, Google Docs).
- **Streaming Applications:** Netflix (Node handles the vast amount of stream requests).
- **Single Page Applications (SPAs):** React/Angular backends.
- **Microservices:** Lightweight architecture.

1.4.3 When NOT to Use Node.js (Red Zone)

- **CPU Heavy Computation:** AI/Machine Learning model training, Video Encoding, complex mathematical calculations.
    - *Reason:* Because Node is single-threaded, a heavy calculation blocks the main thread, freezing the server for *all* clients.

1.4.4 Real-World Case Studies

- **Netflix:** Switched from Java to Node.js to handle their massive streaming interface. Resulted in 70% reduction in startup time.
- **Uber:** Uses Node.js for its massive reliable matching system (Driver <-> Rider) because of its ability to process information quickly and asynchronously.

1.5 Installation & Setup

1.5.1 Installation Steps (Windows)

1. **Download:** Go to nodejs.org. Download the **LTS (Long Term Support)** version. *Exam Tip: Always recommend LTS for stability, Current for latest features.*
2. **Wizard:** Run the `.msi` installer. Ensure "Add to PATH" is checked.
3. **Verify:** Open Command Prompt (`cmd`) and type:

```
node -v
npm -v
```

1.5.2 Installation Steps (macOS)

1. **Homebrew (Recommended):** If you have Homebrew installed:

```
brew install node
```

2. **Manual:** Download `.pkg` from nodejs.org and follow prompts.

1.5.3 The Development Environment

- **IDE:** Visual Studio Code (Industry Standard).
- **Extensions:** ESLint, Prettier.
- **Project Initialization:**

```
mkdir my-project
cd my-project
npm init -y  # Creates package.json with default settings
```

1.6 Practical: REPL and Console

1.6.1 The REPL (Read-Eval-Print Loop)

The REPL is Node's interactive shell, useful for testing quick snippets of code without creating a file.

- **Access:** Type `node` in your terminal.

- **Commands:**
  - `> 10 + 20` -> Returns `30`
  - `> .help` -> Shows dot commands.
  - `> .break` -> Exits multi-line entry.
  - `> .editor` -> Enter editor mode for writing multi-line functions.
  - `> .exit` -> Closes REPL (or Ctrl+C twice).
  - `_` (Underscore) -> Gets the result of the *last* operation.

```
> 5 + 5
10
> _ + 5
15
```

1.6.2 The Console Object

Node's console is similar to the browser's but writes to `stdout` and `stderr`.

Worked Examples:

Example 1: Basic Logging

```
console.log("Hello %s", "World"); // String substitution
```

Example 2: Performance Timing (Critical for debugging)

```
console.time("LoopTime");
for(let i = 0; i < 1000000; i++) {
    // heavy logic
}
console.timeEnd("LoopTime");
// Output: LoopTime: 4.234ms
```

Example 3: Trace

```
// Prints stack trace to see how code reached this point
console.trace("Here is the path");
```

1.7 Exam Strategies & Unit Summary

1.7.1 Revision Cheat Sheet

Concept	Key phrase to remember
V8 Engine	Google's engine, compiles JS to Machine Code.
Libuv	C++ library dealing with OS, File System, Networking.
Non-blocking	"Don't call us, we'll call you" (Callbacks).
Single-Threaded	One waiter, many tables.
REPL	Read, Eval, Print, Loop.

1.7.2 Common Exam Mistakes

1. **Confusing Parallelism with Concurrency:** Node is concurrent (handles many things at once via switching) but not parallel (executing two things at the exact same instant on one thread), unless you use Worker Threads (advanced topic).
2. **Forgetting** `npm init` **:** In practical exams, students often start writing code without initializing the project, leading to "Module not found" errors when they try to install packages.
3. **Blocking the Loop:** Writing `while(true)` or massive calculations in the main thread during practicals. This will crash/freeze your server.

1.8 Practice Questions

Section A: Multiple Choice (1 Mark Each)

1. **Which engine powers Node.js?** a) SpiderMonkey b) V8 c) Chakra d) JVM *Answer: b*
2. **Node.js is best suited for:** a) CPU-intensive applications b) I/O-intensive applications c) Multi-threaded gaming engines d) Graphical editing software *Answer: b*
3. **Which command opens the REPL?** a) `node start` b) `node run` c) `node` d) `npm start` *Answer: c*

Section B: Short Answer (5 Marks)

1. **Explain the role of the** `package.json` **file.** *Hint:* Mention metadata, dependencies list, and scripts.
2. **Differentiate between** `console.log()` **and** `console.error()` **in Node.js.** *Hint:* `stdout` vs `stderr` .
3. **What does the underscore** `_` **variable do in the REPL?**

Section C: Long Answer / Coding (10 Marks)

Q1: "Node.js is single-threaded, yet it handles high concurrency." Defend this statement with a detailed explanation of the Event Loop architecture. *Model Answer Structure:*

- **Introduction:** Define Single-threaded nature.
- **The Mechanism:** Explain the Call Stack (execution), Node APIs (offloading work), and the Callback Queue.
- **The Event Loop:** Describe it as the "traffic controller" checking if the Stack is empty to push callbacks from the Queue.
- **Analogy:** Use the Waiter/Kitchen analogy.

Q2: Write a script that uses `console.time()` to measure how long it takes to sum numbers from 1 to 1 million. Use `console.table()` to display a dummy list of students.

```
// Solution
console.time("SumTimer");

let sum = 0;
for (let i = 1; i <= 1000000; i++) {
    sum += i;
}

console.timeEnd("SumTimer");
console.log(`The sum is: ${sum}`);

const students = [
    { name: "Alice", score: 90 },
    { name: "Bob", score: 85 },
];
console.table(students);
```

Module 2: Node.js Modules and Node Package Manager (NPM)

Comprehensive Exam-Prep Handbook

Professor's Note: *This module is the backbone of Node.js development. If you do not understand how Node handles modules and binary data, you will write inefficient, insecure code. We are not just learning syntax here; we are learning the architecture. Read the "Under the Hood" sections carefully—these distinguish A-grade students from the rest.*

Unit 1: Functions in JavaScript (The Node.js Context)

In Node.js, functions are not just subroutines; they are first-class citizens and the primary mechanism for handling asynchronous operations. You cannot master Node without mastering callbacks and closures.

1.1 First-Class Citizens & Higher-Order Functions

Definition: A language treats functions as "first-class" if they can be assigned to variables, passed as arguments, and returned from other functions.

Why it matters for Node: Node.js is event-driven. You rarely wait for a function to finish; instead, you pass a function (callback) to be executed *later*.

Worked Example 1: Function Expression vs. Declaration

```
// Function Declaration (Hoisted - risky if you rely on order)
function standardLog(msg) {
    console.log(`[LOG]: ${msg}`);
}

// Function Expression (Not hoisted - forces structure)
const errorLog = function(msg) {
    console.error(`[ERR]: ${msg}`);
};

// Passing functions as arguments (Higher-Order)
function logger(message, logFunction) {
    logFunction(message);
}

logger("Server started", standardLog); // Output: [LOG]: Server started
logger("Connection failed", errorLog); // Output: [ERR]: Connection failed
```

1.2 Callbacks and Asynchronous Flow

Definition: A callback is a function passed into another function to be invoked at a later time.

Real-World Application: Reading a file from a database. Node won't stop the server while the disk spins; it registers a callback and moves on.

Worked Example 2: The Callback Pattern

```
const fs = require('fs');

// BAD: Synchronous (Blocks the Event Loop - Don't do this in production)
// const data = fs.readFileSync('data.txt');

// GOOD: Asynchronous (Non-blocking)
console.log("1. Start reading");

fs.readFile('data.txt', 'utf8', (err, data) => {
    if (err) {
        console.error("Error reading file");
        return;
    }
    console.log("3. File content received"); // This runs last
});

console.log("2. End of script");
// Order of execution: 1 -> 2 -> 3
```

1.3 Arrow Functions

Introduced in ES6, these provide a concise syntax and, crucially, **do not bind their own** `this` .

Common Mistake: Using arrow functions in object methods or event handlers where you need dynamic `this` context.

Unit 2: Introduction to Buffers

Hard Truth: JavaScript was originally designed for strings (text). It was terrible at handling binary data (images, video streams, compressed files). Node.js introduced `Buffer` to solve this.

2.1 What is a Buffer?

A `Buffer` is a chunk of memory allocated outside the V8 JavaScript engine. It stores raw binary data as a sequence of integers (0-255), mimicking an array of integers.

Theory:

- **Fixed Size:** Once created, you cannot resize a buffer.
- **Global:** You don't need to `require('buffer')` .

2.2 Worked Examples: Buffer Manipulation

Example 1: Creating Buffers

```
// Allocating 10 bytes (Initialized to zero - SAFER)
const buf1 = Buffer.alloc(10);

// Allocating 10 bytes (Uninitialized - FASTER but UNSAFE)
```

```
// Contains potentially sensitive data from old memory. Avoid unless necessary.
const buf2 = Buffer.allocUnsafe(10);

// Creating from a string
const buf3 = Buffer.from('NodeJS');
console.log(buf3); // <Buffer 4e 6f 64 65 4a 53> (Hex representation)
```

Example 2: Writing and Reading

```
const buf = Buffer.alloc(20);
const len = buf.write("Learn Node");

console.log(`Bytes written: ${len}`);
console.log(buf.toString('utf8', 0, len)); // Decode only written bytes
```

Example 3: Buffer Concatenation (Crucial for Streams)

When receiving data chunks from a network, you get pieces. You need to glue them together.

```
const chunk1 = Buffer.from('Hello ');
const chunk2 = Buffer.from('World');

const total = Buffer.concat([chunk1, chunk2]);
console.log(total.toString()); // Hello World
```

Unit 3: The Node.js Module System

Node.js uses the **CommonJS** module system. Every file in Node is treated as a separate module. This isolation prevents variable pollution (variables in `a.js` don't overwrite variables in `b.js`).

3.1 The `require()` Function: Under the Hood

When you write `const lib = require('./lib')` , Node performs three steps:

1. **Resolving:** Finds the absolute path of the file.
2. **Loading:** Reads the file content.
3. **Wrapping:** Wraps the code in an IIFE (Immediately Invoked Function Expression).

The Wrapper Function: Your code is actually executed inside this:

```
(function(exports, require, module, __filename, __dirname) {
    // Your code lives here
});
```

This is why `__dirname` and `module` are available without defining them.

3.2 Core Modules

Built-in modules that come with the Node binary. You import them by name.

Module	Purpose	Common Functions
`fs`	File System	`readFile` , `writeFile` , `stat`
`path`	File Paths	`join` , `resolve` , `extname`
`os`	Operating System	`cpus` , `freemem` , `userInfo`
`http`	Networking	`createServer` , `request`
`events`	Event Emitter	`on` , `emit`

Worked Example: Using `path` and `os`

```
const path = require('path');
const os = require('os');

const filename = path.join(__dirname, 'files', 'report.txt');
console.log(`Saving to: ${filename}`);
console.log(`System Uptime: ${os.uptime()} seconds`);
```

3.3 Local Modules & Exports

You create local modules to organize your code.

Pattern A: Exporting Properties (The `exports` shortcut)

```
// math.js
exports.add = (a, b) => a + b;
exports.sub = (a, b) => a - b;

// main.js
const math = require('./math');
console.log(math.add(5, 3));
```

Pattern B: Exporting a Class or Object (The `module.exports` override)

```
// logger.js
class Logger {
    log(msg) { console.log(msg); }
}
module.exports = Logger; // Overwrites the exports object entirely

// main.js
```

```
const Logger = require('./logger');
const dbLogger = new Logger();
```

Common Pitfall (The Broken Link): `exports` is just a reference to `module.exports`. If you reassign `exports`, you break the link.

```
exports = { func: () => {} }; // WRONG! Does not export anything.
module.exports = { func: () => {} }; // CORRECT.
```

Unit 4: Node Package Manager (NPM)

NPM is two things: a repository (registry) of code, and a CLI tool to manage that code.

4.1 package.json

The manifest file for your project. To create one: `npm init` or `npm init -y` (defaults).

Key Fields:

- `scripts`: Aliases for terminal commands (e.g., `"start": "node index.js"`).
- `dependencies`: Libraries required for the app to *run* (e.g., Express).
- `devDependencies`: Libraries required only to *develop/test* (e.g., Jest, Nodemon).

4.2 Semantic Versioning (SemVer)

Versions are `Major.Minor.Patch` (e.g., `1.4.2`).

- **Patch:** Bug fixes (Backward compatible).
- **Minor:** New features (Backward compatible).
- **Major:** Breaking changes.

Dependencies Syntax:

- `"express": "4.17.1"` -> Exact version.
- `"express": "~4.17.1"` -> Allows Patch updates (4.17.x).
- `"express": "^4.17.1"` -> Allows Minor updates (4.x.x). **Default behavior.**

4.3 Local vs. Global Installation

- **Local (** `npm install lodash` **):** Installs in `./node_modules`. Accessible only to this project via `require()`. **Use this for 99% of libraries.**
- **Global (** `npm install -g nodemon` **):** Installs in system directory. Accessible from command line anywhere. **Use only for CLI tools.**

Updating Packages:

- `npm outdated`: Checks for newer versions.

- `npm update` : Updates packages strictly based on the semver rules in `package.json` .

Practical: Mini-Case Study

Scenario: You need a script that takes a user's name, converts it to uppercase using an external library (`lodash`), creates a buffer of the name, saves it to a file, and logs the OS architecture.

Solution:

1. **Setup:**

```
npm init -y
npm install lodash
```

2. **Code (`index.js`):**

```
const fs = require('fs');
const os = require('os');
const path = require('path');
const _ = require('lodash'); // External Module

const username = "john doe";

// 1. Use external module
const formatted = _.toUpper(username);

// 2. Use Buffer
const dataBuffer = Buffer.from(formatted);

// 3. Use Core Modules (path, fs)
const filePath = path.join(__dirname, 'user_data.bin');

fs.writeFile(filePath, dataBuffer, (err) => {
    if (err) throw err;
    console.log(`Saved ${dataBuffer.length} bytes to ${filePath}`);
    console.log(`Server Architecture: ${os.arch()}`);
});
```

Exam Strategies & Revision

Cheat Sheet: Commands

Command	Action
`npm init`	Create package.json
`npm i <pkg>`	Install locally and add to dependencies
`npm i -D <pkg>`	Install as devDependency

`npm i -g <pkg>`	Install globally
`npm uninstall <pkg>`	Remove package
`node_modules`	Folder where packages live (NEVER commit this to Git)

Common Mistakes (Exam Traps)

1. **Confusing `exports` and `module.exports`:** Remember, if you want to export a single class or function directly, use `module.exports = ...`. If you want a collection of functions, `exports.funcName = ...` is fine.
2. **Sync vs Async:** Using `fs.readFileSync` in a server endpoint question. **Immediate FAIL.** Always use the callback version `fs.readFile` or Promises in server contexts to avoid blocking the event loop.
3. **Pathing:** Assuming a file is in the current directory (`./file.txt`). Always use `path.join(__dirname, 'file.txt')` for robustness.

Question Bank

Section A: Multiple Choice (2 Marks Each)

Q1. Which Core Module is used to handle file paths in a cross-platform way? a) `fs` b) `http` c) `path` d) `url` *Answer: c) path*

Q2. What does the caret symbol (`^`) signify in `package.json` versioning? a) Exact version only. b) Updates to the latest Major version. c) Updates to the latest Minor version. d) Updates to the latest Patch version only. *Answer: c) Updates to the latest Minor version.*

Q3. Buffers are stored in: a) The V8 Heap b) Outside the V8 Heap c) LocalStorage d) The Stack *Answer: b) Outside the V8 Heap*

Section B: Short Answer (5 Marks)

Q. Explain the purpose of `package-lock.json`. Why must it be committed to version control? **Model Answer:** `package-lock.json` describes the exact tree of dependencies that were generated when `npm install` was last run. While `package.json` might allow fuzzy versions (e.g., `^1.0.0`), `package-lock.json` locks down the specific version installed (e.g., `1.0.4`) and the versions of its sub-dependencies. It must be committed to Git to ensure "reproducible builds." This guarantees that every developer and the production server install exactly the same dependencies, preventing "it works on my machine" bugs.

Section C: Coding/Long Answer (10 Marks)

Q. Create a local module named `calculator.js` that exports functions for `add` and `multiply`. Write a `main.js` file that imports this module. The `main.js` should also use the `fs` module to append the result of a multiplication operation to a file named `history.txt`.

Solution:

```
// calculator.js
const add = (a, b) => a + b;
const multiply = (a, b) => a * b;

module.exports = {
    add,
    multiply
};

// main.js
const calc = require('./calculator');
const fs = require('fs');

const result = calc.multiply(10, 5);
const logMessage = `Multiplication Result: ${result}\n`;

fs.appendFile('history.txt', logMessage, (err) => {
    if (err) console.error("Failed to write to file");
    else console.log("Result saved to history.txt");
});
```

End of Module 2 Handbook

Module 3: Creating a Basic Web Server & File System Operations

Lecture Hours: 6H | **Practical Hours:** 14H **Prerequisites:** JavaScript Fundamentals, Basic Networking Concepts.

1. Introduction: The Heart of Node.js

This module covers the two most critical built-in libraries in Node.js: `http` and `fs` .

The Reality Check: If you can't write a raw HTTP server without Express.js, you don't understand how the web works. Frameworks are crutches until you understand the underlying mechanics. Similarly, if you use synchronous file operations (`readFileSync`) in a production web server, you are blocking the Event Loop and effectively killing your server's concurrency.

This chapter will force you to understand the *low-level* operations of handling requests and manipulating files.

2. The HTTP Module

Node.js uses the `http` module to transfer data over the Hyper Text Transfer Protocol (HTTP). Unlike PHP or ASP.NET, which often require a separate web server (like Apache or IIS), Node.js **is** the web server.

2.1 Anatomy of the HTTP Transaction

To pass the exam, you must be able to diagram this flow:

1. **Client** sends a Request (Method, URL, Headers, Body).
2. **Server** receives the Request (via `http.createServer`).
3. **Server** processes the Request (Parsing, Database, Logic).
4. **Server** sends a Response (Status Code, Headers, Body).

2.2 Creating a Server

The `http.createServer()` method includes a callback function requestListener that is automatically added to the `'request'` event.

Syntax:

```
const http = require('http');

const server = http.createServer((req, res) => {
    // Logic goes here
});

server.listen(3000, () => {
    console.log('Server running on port 3000');
```

```
});
```

Detailed Breakdown of Arguments:

- `req` **(IncomingMessage):** A Readable Stream. Contains details about what the client wants.
  - `req.url` : The path requested (e.g., `/home`).
  - `req.method` : The verb used (GET, POST, etc.).
  - `req.headers` : Meta-data about the request.
- `res` **(ServerResponse):** A Writable Stream. Used to send data back to the client.
  - `res.writeHead()` : Writes status and headers.
  - `res.write()` : Sends chunks of the body.
  - `res.end()` : Signals communication is over.

2.3 Worked Examples: HTTP

Example 1: The "Hello World" API (Basic)

Objective: Return a simple text response.

```
const http = require('http');

const server = http.createServer((req, res) => {
    // 1. Set the Header (MIME type is crucial)
    res.setHeader('Content-Type', 'text/plain');

    // 2. Set Status Code (200 = OK)
    res.statusCode = 200;

    // 3. Send Body and End
    res.end('Hello from Node.js Server!');
});

server.listen(3000);
```

Example 2: JSON API with Routing (Intermediate)

Objective: Handle different URLs and return JSON data.

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.setHeader('Content-Type', 'application/json');

    if (req.url === '/api/users' && req.method === 'GET') {
        const users = [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }];
        res.writeHead(200);
```

```
        res.end(JSON.stringify(users));
    } else if (req.url === '/') {
        res.writeHead(200);
        res.end(JSON.stringify({ message: 'Welcome to the API' }));
    } else {
        // 404 Handling
        res.writeHead(404);
        res.end(JSON.stringify({ error: 'Route not found' }));
    }
});

server.listen(3000);
```

Example 3: Sending a Request (Acting as a Client)

The syllabus explicitly mentions "sending requests to the server". This typically refers to `http.request`.

```
const http = require('http');

const options = {
  hostname: 'jsonplaceholder.typicode.com',
  path: '/todos/1',
  method: 'GET'
};

const req = http.request(options, (res) => {
  console.log(`STATUS: ${res.statusCode}`);

  res.on('data', (chunk) => {
    console.log(`BODY: ${chunk}`);
  });
});

req.on('error', (e) => {
  console.error(`Problem with request: ${e.message}`);
});

req.end(); // Must be called to finalize the request
```

3. The File System (fs) Module

The `fs` module allows you to work with the file system on your computer.

3.1 The Golden Rule: Sync vs. Async

Exam Warning: If you write `fs.readFileSync` in an exam question asking for a "Scalable Web Server," I will deduct marks.

- **Synchronous (`readFileSync`):** Blocks the thread. The entire server stops and waits for the hard drive to spin and read the file. No other user can be served during this time.

- **Asynchronous (`readFile`):** Non-blocking. Node.js offloads the I/O task to the system kernel and continues executing code. When the file is ready, a callback fires.

3.2 Reading Files

Synchronous Reading (Use only for scripts/config loading)

```
const fs = require('fs');

try {
    const data = fs.readFileSync('input.txt', 'utf8');
    console.log(data);
} catch (err) {
    console.error(err);
}
```

Asynchronous Reading (The Node.js Way)

```
const fs = require('fs');

// 'utf8' argument converts buffer to string automatically
fs.readFile('input.txt', 'utf8', (err, data) => {
    if (err) {
        console.error('Error reading file:', err);
        return;
    }
    console.log('File content:', data);
});
console.log('This prints BEFORE the file content.');
```

3.3 Writing and Updating Files

Operation	Method	Behavior
Write	`fs.writeFile()`	Replaces the file if it exists; creates it if it doesn't.
Append	`fs.appendFile()`	Adds data to the end of the file.
Delete	`fs.unlink()`	Removes the file from the filesystem.

Example: Log Recorder (Write + Append)

```
const fs = require('fs');

const logMessage = `Log Entry: ${new Date().toISOString()}\n`;

// 1. Append to log.txt
fs.appendFile('server.log', logMessage, (err) => {
```

```
    if (err) throw err;
    console.log('Log updated!');
});
```

3.4 Deleting Files

```
fs.unlink('temp.txt', (err) => {
    if (err) {
        console.error('File not found or locked');
        return;
    }
    console.log('File deleted successfully');
});
```

4. Case Study: A Static File Server

Combining `http` and `fs` modules.

Scenario: You need to serve HTML, CSS, and Image files based on the user's browser request. This is the foundation of how servers like Apache or Nginx work essentially.

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
    // 1. Build file path
    let filePath = path.join(__dirname, 'public', req.url === '/' ? 'index.html'

    // 2. Determine extension for MIME type
    const extname = path.extname(filePath);
    let contentType = 'text/html';

    switch (extname) {
        case '.js': contentType = 'text/javascript'; break;
        case '.css': contentType = 'text/css'; break;
        case '.json': contentType = 'application/json'; break;
        case '.png': contentType = 'image/png'; break;
    }

    // 3. Read File
    fs.readFile(filePath, (err, content) => {
        if (err) {
            if (err.code == 'ENOENT') {
                // Page not found
                fs.readFile(path.join(__dirname, 'public', '404.html'), (error, c
                    res.writeHead(404, { 'Content-Type': 'text/html' });
                    res.end(content, 'utf-8');
                });
            } else {
                // Server Error
                res.writeHead(500);
                res.end(`Server Error: ${err.code}`);
```

```
            }
        } else {
            // Success
            res.writeHead(200, { 'Content-Type': contentType });
            res.end(content, 'utf-8');
        }
    });
});

server.listen(5000, () => console.log('Static Server running on port 5000'));
```

5. Exam Preparation & Strategies

5.1 Cheat Sheet: Key Methods

Module	Method	Purpose	Key Note
HTTP	`http.createServer(cb)`	Creates a server instance	Callback runs on *every* request.
HTTP	`res.writeHead(code, headers)`	Sends status and headers	Must run before `res.end()`.
FS	`fs.readFile(path, enc, cb)`	Reads file asynchronously	`enc` (e.g., 'utf8') is optional but recommended for text.
FS	`fs.writeFile(path, data, cb)`	Overwrites file	Creates file if missing.
FS	`fs.unlink(path, cb)`	Deletes file	Irreversible.

5.2 Common Student Mistakes (Don't do these)

1. **Forgetting `res.end()`:** The client's browser will spin forever because the server never signaled the response is complete.
2. **Writing Headers Twice:** You cannot use `res.setHeader` or `res.writeHead` after `res.write()` or `res.end()` has sent the first chunk of data.
3. **Sync inside Async:** Using `readFileSync` inside the `http.createServer` callback. This is a performance disaster.
4. **Ignoring Errors:** In `fs.readFile`, always check `if (err)` before trying to use `data`.

5.3 Practice Questions

Section A: Multiple Choice (2 Marks Each)

1. Which method is used to signal the server that all response headers and body have been sent? a) `res.send()` b) `res.stop()` c) `res.end()` d) `res.finish()`

2. What happens if you use `fs.writeFile` on a file that already exists? a) It throws an error. b) It appends the data to the end. c) It replaces the old content entirely. d) It creates a backup file.

3. In the HTTP module, the `req` object is an instance of: a) `http.ServerResponse` b) `http.IncomingMessage` c) `fs.ReadStream` d) `http.ClientRequest`

(Answers: 1-c, 2-c, 3-b)

Section B: Short Answer (5 Marks)

Q1. Explain the difference between `fs.readFile` and `fs.readFileSync`. Why is the asynchronous version preferred in Node.js applications? *Hint: Discuss "Event Loop Blocking" and "Concurrency". Draw a timeline comparison.*

Q2. Write a code snippet to delete a file named `database.lock`. Handle the error if the file does not exist.

Section C: Long Answer / Coding (10 Marks)

Q3. Scenario: Create a Node.js web server that:

1. Listens on port 8080.
2. On `GET /log`, reads a file named `system.txt` and sends the content to the browser.
3. On `POST /log`, receives text data from the request body and appends it to `system.txt`.
4. Handles errors (e.g., file not found).

Solution Outline:

- Import `http` and `fs`.
- Check `req.method`.
- If GET: `fs.readFile -> res.write -> res.end`.
- If POST: Listen for `req.on('data')` to collect body, then `fs.appendFile`.

6. Historical Context & Modern Application

- **History:** Before Node.js (2009), I/O was typically blocking (e.g., in early Java/PHP setups, one thread per request). Node.js introduced the reactor pattern to server-side JS, allowing a single thread to handle thousands of concurrent connections using non-blocking I/O.
- **Modern Usage:** While we use `fs` directly for tools and scripts, modern web frameworks like **Express** or **Fastify** wrap the `http` module to make routing easier. However, `fs`

Module 4: Debugging, Events, and Express.js

Comprehensive Exam Preparation Handbook

Author: Prof. Gemini

Course: Advanced Web Technologies

Target Length: Section contributes ~25 pages to the full handbook.

Table of Contents

1. **Unit 1: Debugging Node.js Applications**
    - 1.1 The Theory of Debugging in V8
    - 1.2 Core Node.js Debugger (CLI)
    - 1.3 Visual Studio Code Debugging
    - 1.4 Worked Examples & Case Studies
2. **Unit 2: The Event-Driven Architecture**
    - 2.1 Understanding the `EventEmitter` Class
    - 2.2 Design Patterns: Returning vs. Inheriting
    - 2.3 Memory Leaks and Best Practices
    - 2.4 Worked Examples
3. **Unit 3: Building with Express.js**
    - 3.1 The Express Philosophy & Routing
    - 3.2 Middleware: The Backbone of Express
    - 3.3 Building a Basic Web Application
    - 3.4 Worked Examples
4. **Exam Preparation Suite**
    - 4.1 Quick Revision Cheat Sheets
    - 4.2 Practice Questions (MCQ, Short, Long)
    - 4.3 Model Answers & Grading Rubric

Unit 1: Debugging Node.js Applications

1.1 Historical Context and Theory

In the early days of JavaScript, debugging often meant peppering code with `console.log()` statements. While effective for simple scripts, this approach fails in complex, asynchronous Node.js applications where the order of execution isn't linear.

Node.js uses the **V8 Inspector Protocol**. This is a protocol that allows external tools (clients) to communicate with the V8 engine to inspect state, pause execution, and analyze the heap. When you start Node with a debug flag, it opens a WebSocket port (usually 9229) that tools like Chrome DevTools or VS Code can attach to.

1.2 The Core Node.js Debugger (CLI)

Node.js ships with an out-of-the-box command-line debugger. It is austere but powerful, useful when you are debugging on a remote server via SSH where a GUI isn't available.

Command:

```
node inspect app.js
```

Key Commands Table:

Command	Description	Equivalent in GUI
`cont` or `c`	Continue execution until the next breakpoint.	Play button
`next` or `n`	Step over the next function call.	Step Over
`step` or `s`	Step *into* the function call.	Step Into
`out` or `o`	Step *out* of the current function.	Step Out
`repl`	Enter a Read-Eval-Print-Loop to inspect variables.	Debug Console

1.3 Debugging with Visual Studio Code

VS Code is the industry standard for Node.js development. Its debugger wraps the V8 protocol in a user-friendly UI.

The `launch.json` Configuration: To debug effectively, you must understand the configuration file found in `.vscode/launch.json`.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Program",
      "skipFiles": ["<node_internals>/**"],
      "program": "${workspaceFolder}/app.js"
    }
  ]
```

```
}
```

- `request` : "launch" starts the app; "attach" connects to an already running app.
- `skipFiles` : Crucial for skipping Node's internal core modules (like `fs.js` or `net.js`) when stepping through code, so you focus only on *your* code.

1.4 Worked Examples: Debugging Scenarios

Scenario 1: The Infinite Loop *Problem:* The server hangs and CPU spikes to 100%. *Code:*

```
function calculateFactorial(n) {
    if (n === 0) return 1;
    return n * calculateFactorial(n); // BUG: n never decrements
}
```

Debugging Step:

1. Set a breakpoint on the `return` line.
2. Watch the 'Call Stack' in VS Code.
3. *Observation:* You will see the stack grow indefinitely (`calculateFactorial` stacking on top of `calculateFactorial`) until `RangeError: Maximum call stack size exceeded` .
4. *Fix:* Change recursive call to `calculateFactorial(n - 1)` .

Scenario 2: Asynchronous Confusion *Problem:* Variable is `undefined` because the developer treated an async operation as synchronous. *Code:*

```
let user;
setTimeout(() => { user = "Alice"; }, 100);
console.log(user.toUpperCase()); // CRASH: Cannot read property of undefined
```

Debugging Step:

1. Run with `node inspect` .
2. Execution stops at `console.log` .
3. Type `repl` and check `user` . It is `undefined` .
4. *Analysis:* The `setTimeout` callback hasn't fired yet. The main thread continued immediately.

Unit 2: The Event-Driven Architecture

2.1 Understanding `EventEmitter`

Node.js is asynchronous and event-driven. The `events` module is the skeleton key to this architecture. The `EventEmitter` class allows objects to subscribe to ("listen for") and publish

("emit") named events.

Anatomy of an Emitter:

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// 1. Subscribe
myEmitter.on('orderPlaced', (id) => {
    console.log(`Processing order #${id}`);
});

// 2. Publish
myEmitter.emit('orderPlaced', 101);
```

2.2 Returning Emitters vs. Inheriting

There are two primary ways to use events in your modules.

Approach A: Returning an Emitter (Composition) Used when you just need simple notification capabilities.

```
function createTicker() {
    const emitter = new EventEmitter();
    setInterval(() => emitter.emit('tick'), 1000);
    return emitter;
}
```

Approach B: Inheriting from EventEmitter (Inheritance) Used when your class *is* an entity that naturally produces events (e.g., a File Stream, a Server).

```
class PizzaShop extends EventEmitter {
    constructor() {
        super();
        this.orderNumber = 0;
    }

    order() {
        this.orderNumber++;
        this.emit('order', this.orderNumber);
    }
}
```

Professor's Note: Inheritance is generally preferred for creating robust modules (like typical `Module 4` exam questions ask for) because it allows the object to possess its own methods *and* event capabilities simultaneously.

2.3 Memory Leaks: The Silent Killer

If you register an event listener `.on()` but never remove it, and the object persists, the listener stays in memory.

- **Warning Sign:** Node prints `MaxListenersExceededWarning` (default limit is 10).
- **Solution:** Use `.once()` for one-time events or `.removeListener()` / `.off()` when the event is no longer needed.

Unit 3: Building with Express.js

3.1 Configuring Routes

Express is a "minimalist" framework. It doesn't force a structure, but it provides a powerful routing API.

Route Parameters vs. Query Strings:

- **Route Params (`/users/:id`):** Used to identify a specific resource. Mandatory.
  - `GET /users/42 -> req.params.id = 42`
- **Query Strings (`/users?sort=desc`):** Used to filter or sort resources. Optional.
  - `GET /users?sort=asc -> req.query.sort = 'asc'`

3.2 Middleware: The "Onion" Architecture

Middleware functions have access to the request object (`req`), the response object (`res`), and the next function in the application's request-response cycle (`next`).

Think of Middleware as layers of an onion. The request goes *through* the layers to the core (the route handler), and the response goes *back out*.

Types of Middleware:

1. **Application-level:** `app.use()`
2. **Router-level:** `router.use()`
3. **Built-in:** `express.json()` , `express.static()`
4. **Error-handling:** Defined with 4 arguments `(err, req, res, next)` .

3.3 Worked Examples: Express Structures

Example: A Middleware that logs time

```
const express = require('express');
const app = express();

// Middleware definition
const requestTime = function (req, res, next) {
  req.requestTime = Date.now();
  next(); // CRITICAL: Must call next() or request hangs
};

app.use(requestTime);
```

```
app.get('/', (req, res) => {
  let responseText = 'Hello World!<br>';
  responseText += '<small>Requested at: ' + req.requestTime + '</small>';
  res.send(responseText);
});
```

Unit 4: Exam Preparation Suite

4.1 Revision Cheat Sheet

Concept	Command / Code	Purpose
Inspect	`node --inspect app.js`	Enable V8 inspector protocol.
Debugger	`debugger;`	Hardcoded breakpoint in JS code.
Emit	`emitter.emit('name', data)`	Trigger an event.
Listen	`emitter.on('name', fn)`	Handle an event repeatedly.
Once	`emitter.once('name', fn)`	Handle an event exactly once.
Express Param	`app.get('/user/:id')`	Define a dynamic route segment.
Wildcard	`app.get('*')`	Catch-all route (usually 404).
Middleware	`app.use((req, res, next) => ...)`	Function run before route handler.

4.2 Practice Questions

Section A: Multiple Choice (1 Mark Each)

Q1: Which command is used to step *over* the next function call in the Node.js CLI debugger? A) `sb` B) `next` C) `cont` D) `step` *Answer: B (next)*

Q2: If a `try/catch` block is missing in a synchronous Express route handler, what happens when an error is thrown? A) The server crashes. B) Express catches it automatically and sends a 500. C) The client waits indefinitely. D) The error is ignored. *Answer: B (Express 4.x catches synchronous errors automatically. Note: It does NOT catch async errors automatically unless using Express 5.x).*

Q3: Why might you use `emitter.once()` instead of `emitter.on()` ? A) It is faster. B) To prevent memory leaks for events that only happen during initialization. C) `on` is deprecated. D) `once` allows multiple arguments. *Answer: B*

Section B: Short Answer (5 Marks Each)

Q4: Explain the difference between `application/x-www-form-urlencoded` and `application/json` in the context of Express middleware. *Hint:* Discuss `express.json()` vs `express.urlencoded()` and how HTML forms submit data vs how AJAX/React apps submit data.

Q5: In the context of the Observer pattern, explain how Node.js `EventEmitter` decouples the producer of an event from the consumer. Provide a real-world analogy. *Hint:* Use the analogy of a Radio Station (Emitter) and Listeners. The station broadcasts regardless of who is listening. The listeners tune in without affecting the station's broadcast schedule.

Section C: Long Answer / Coding (10 Marks Each)

Q6: Debugging Scenario You have an Express route designed to fetch user data from a database and return it. However, the request hangs indefinitely—the browser spinner just keeps spinning.

1. List three possible reasons for this behavior.
2. Write the code for a Middleware function that would help you debug this by logging the exact URL and timestamp of every incoming request.

Model Answer Strategy for Q6:

1. *Reasons:*
   - A middleware forgot to call `next()`.
   - The database query is asynchronous but the callback/promise chain is broken (never calling `res.send`).
   - An exception occurred in an async block without a `catch`, causing an unhandled rejection that didn't crash the process but stopped the response.
2. *Code:*

```
app.use((req, res, next) => {
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
    next();
});
```

Q7: Building a Robust Event-Driven Module Write a Node.js class named `StockTicker` that inherits from `EventEmitter`.

1. It should accept a `symbol` (e.g., "AAPL") in the constructor.
2. It should have a method `startMonitoring()` that emits a `priceUpdate` event with a random price every 2 seconds.
3. Write a consumer script that listens to this event and prints the price.
4. Ensure that if the price drops below 10, it emits a `crash` event and stops monitoring.

Model Answer Code:

```
const EventEmitter = require('events');

class StockTicker extends EventEmitter {
    constructor(symbol) {
        super();
        this.symbol = symbol;
        this.intervalId = null;
    }

    startMonitoring() {
        this.intervalId = setInterval(() => {
            const price = Math.floor(Math.random() * 100);
            this.emit('priceUpdate', { symbol: this.symbol, price });

            if (price < 10) {
                this.emit('crash', price);
                this.stopMonitoring();
            }
        }, 2000);
    }

    stopMonitoring() {
        if (this.intervalId) {
            clearInterval(this.intervalId);
            this.emit('end');
        }
    }
}

// Consumer
const apple = new StockTicker('AAPL');
apple.on('priceUpdate', (data) => console.log(`${data.symbol}: $${data.price}`));
apple.on('crash', (price) => console.log(`MARKET CRASH DETECTED AT $${price}!`));
apple.startMonitoring();
```

4.3 Practical Lab Guide (12 Hours)

Lab Exercise 1: The Detective

- **Objective:** Fix a broken Express app.
- **Provided:** A zip file with an app that throws 500 errors on specific routes.
- **Task:** Use VS Code `launch.json` to attach the debugger. Set "Exception Breakpoints" to "Uncaught Exceptions". Find the line causing the crash.

Lab Exercise 2: The Chat Server

- **Objective:** Use Events.
- **Task:** Build a CLI chat where multiple "User" objects inherit from `EventEmitter`. When User A types a message, User B and C should receive it via an event.

Lab Exercise 3: Middleware Chaining

- **Objective:** Authentication simulation.
- **Task:** Create three middleware functions:

1. `checkAuth` : Checks if a header `x-secret` is present.
2. `logAccess` : Writes access attempts to a file.
3. `errorHandler` : Returns a nice JSON error if `checkAuth` fails.

- **Constraint:** Apply `checkAuth` only to `/admin` routes.

Module 5: Serving Static Resources and Database Connectivity

Comprehensive Exam-Prep Handbook

1. Unit Overview & Historical Context

1.1 Historical Context

In the early days of the web (Web 1.0), servers strictly delivered static content—HTML files sitting on a hard drive. As the web evolved into Web 2.0, dynamic content generation became king. However, the need to serve "assets"—images, CSS stylesheets, client-side JavaScript, and fonts—never went away. Today, Node.js and Express.js provide a high-performance, non-blocking way to serve these assets alongside dynamic API routes.

Simultaneously, the data storage landscape shifted. Relational databases (SQL) dominated for decades until the explosion of "Big Data" and unstructured data in the late 2000s gave rise to **NoSQL**. MongoDB emerged as the leader in this space, offering a flexible, document-oriented model that mirrors the JSON objects used in JavaScript, making it the natural database choice for Node.js developers (the "M" in the MERN stack).

1.2 Learning Objectives

By the end of this module, you will be able to:

1. Configure Express.js to serve static assets efficiently.
2. Implement and understand the flow of Middleware functions.
3. Differentiate between SQL and NoSQL databases.
4. Perform full CRUD operations on MongoDB using Node.js.
5. Securely manage database configurations and connection strings.

2. Serving Static Files with Express.js

2.1 Theory and Definitions

Static Files are files that clients download as-is from the server. Examples include images (`.png` , `.jpg`), generic HTML files, CSS frameworks, and browser-side JavaScript libraries.

`express.static` is the only built-in middleware function in Express. It creates a route handler for you that checks a specified directory for the requested file. If found, it streams the file; if not, it calls `next()` to move to the next route handler.

2.2 Worked Examples

Example 1: Basic Static File Serving

Scenario: You have a folder named `public` containing `style.css` and `logo.png` .

```
const express = require('express');
const app = express();
const path = require('path');

// Syntax: app.use(express.static(root, [options]))
app.use(express.static(path.join(__dirname, 'public')));

app.listen(3000, () => console.log('Server running on port 3000'));
```

- **Request:** `GET /style.css`
- **Response:** The server looks inside `public/`, finds `style.css`, and sends it.
- **Note:** The URL does *not* include `/public`. It is relative to the root.

Example 2: Virtual Path Prefix

Scenario: You want the URL to look like `/assets/style.css` but the file is physically inside `public/style.css`.

```
app.use('/assets', express.static(path.join(__dirname, 'public')));
```

- **Request:** `GET /assets/style.css` -> Maps to `public/style.css`.
- **Request:** `GET /style.css` -> **404 Not Found** (because the prefix is missing).

Example 3: Multiple Static Directories

Scenario: You store images in `images/` and scripts in `scripts/`. Express searches directories in the order defined.

```
app.use(express.static(path.join(__dirname, 'images')));
app.use(express.static(path.join(__dirname, 'scripts')));
```

- **Logic:** If a request comes for `avatar.png`, Express checks `images/` first. If found, it sends it. If not, it checks `scripts/`.

Example 4: Absolute Paths (Best Practice)

Critical Concept: Always use `path.join(__dirname, 'folder')`.

- *Why?* If you run node from a different directory (e.g., `node src/app.js`), relative paths like `'./public'` will break. `__dirname` ensures the path is always relative to the file where the code is written.

3. Working with Middleware in Express.js

3.1 Theory: The Request-Response Cycle

Middleware functions are functions that have access to the **request object (** `req` **)**, the **response object (** `res` **)**, and the **next middleware function (** `next` **)**.

Capabilities of Middleware:

1. Execute any code.
2. Make changes to the request and response objects (e.g., parsing JSON).
3. End the request-response cycle (e.g., sending a response).
4. Call the next middleware in the stack.

3.2 Types of Middleware

1. **Application-level:** Bound to `app` using `app.use()` .
2. **Router-level:** Bound to `express.Router()` .
3. **Error-handling:** Defined with 4 arguments `(err, req, res, next)` .
4. **Built-in:** `express.static` , `express.json` .
5. **Third-party:** `morgan` (logging), `cors` (security).

3.3 Worked Examples

Example 1: Application-Level Logger

```
const requestTime = (req, res, next) => {
    req.requestTime = Date.now();
    console.log(`[${new Date().toISOString()}] ${req.method} request to ${req.url
    next(); // CRITICAL: Must call next() or request hangs
};

app.use(requestTime);

app.get('/', (req, res) => {
    res.send(`Request received at: ${req.requestTime}`);
});
```

Example 2: Configurable Guard Middleware (Authentication Stub)

```
const requireApiKey = (req, res, next) => {
    const apiKey = req.query.api_key;
    if (apiKey && apiKey === 'secret123') {
        next(); // Authorized
    } else {
        res.status(401).send('Unauthorized: Invalid API Key'); // Cycle ends here
    }
};

// Protect only this route
app.get('/dashboard', requireApiKey, (req, res) => {
    res.send('Welcome to the Dashboard');
```

```
});
```

Example 3: Error Handling Middleware

Theory: Error handlers *must* be defined last, after other `app.use()` and routes calls.

```
// Normal route that might error
app.get('/broken', (req, res, next) => {
    const err = new Error('Something went wrong!');
    next(err); // Passing an argument to next() signals an error
});

// Error Handler (4 arguments)
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).send('Something broke!');
});
```

4. Introduction to NoSQL & MongoDB

4.1 SQL vs. NoSQL

Feature	Relational (SQL)	NoSQL (MongoDB)
Data Structure	Tables with Rows and Columns	Collections with Documents (JSON-like)
Schema	Rigid (Defined beforehand)	Flexible (Schemaless or Dynamic)
Scaling	Vertical (Bigger CPU/RAM)	Horizontal (Sharding across servers)
Relationships	Complex Joins (Foreign Keys)	Embedded Data or References
Best For	Structured data, complex transactions	Big data, real-time apps, content management

4.2 Why MongoDB?

1. **Document Model:** Maps naturally to objects in code.
2. **High Availability:** Replica sets provide automatic failover.
3. **Horizontal Scalability:** Native sharding.
4. **Rich Query Language:** Supports filtering, aggregation, and text search.

4.3 Use Cases

- **Content Management Systems (CMS):** Storing blog posts with varying structures.

- **IoT:** Storing massive streams of sensor data.
- **Mobile Apps:** Flexible backends that evolve with app updates.
- **Real-time Analytics:** Storing logs and user actions.

5. Connecting Node.js to MongoDB

5.1 The MongoDB Node.js Driver

While libraries like Mongoose are common, the syllabus implies understanding the native driver first.

Prerequisites:

1. MongoDB server running (Locally or Atlas).
2. `npm install mongodb`

5.2 Connection Strings & Configuration

Format: `mongodb://[username:password@]host1[:port1][,...hostN[:portN]][/[defaultauthdb][?options]]`

Security Best Practice: NEVER hardcode credentials. Use Environment Variables.

```
# .env file
DB_CONNECTION_STRING=mongodb://localhost:27017
DB_NAME=university_db
```

5.3 Worked Example: Establishing Connection

```
const { MongoClient } = require('mongodb');
require('dotenv').config(); // Load .env

const url = process.env.DB_CONNECTION_STRING;
const client = new MongoClient(url);
const dbName = process.env.DB_NAME;

async function connectToDb() {
    try {
        await client.connect();
        console.log('Connected successfully to server');
        const db = client.db(dbName);
        return db;
    } catch (error) {
        console.error('Connection failed:', error);
        return null;
    }
}
```

6. CRUD Operations (Create, Read, Update, Delete)

*Note: The syllabus mentions "select command". In MongoDB, this corresponds to **Projections** in the `find` method.*

6.1 Create (Insert)

Methods: `insertOne()`, `insertMany()`

Example:

```
async function addStudents(db) {
    const students = db.collection('students');

    // Insert One
    const resultOne = await students.insertOne({
        name: "Alice",
        gpa: 3.8,
        major: "CS"
    });
    console.log(`Added student with ID: ${resultOne.insertedId}`);

    // Insert Many
    const resultMany = await students.insertMany([
        { name: "Bob", gpa: 3.2, major: "Math" },
        { name: "Charlie", gpa: 3.9, major: "CS" }
    ]);
    console.log(`Added ${resultMany.insertedCount} students`);
}
```

6.2 Read (Find & Select)

Methods: `find()`, `findOne()` **Operators:** `$gt` (greater than), `$lt` (less than), `$in` (in array), `$or` (logical OR).

The "Select" Command (Projection): By default, MongoDB returns all fields. To select specific fields (like SQL `SELECT name, gpa FROM students`), use the second argument in `find`.

Example:

```
async function findHighAchievers(db) {
    const students = db.collection('students');

    // Query: GPA > 3.5
    // Projection: Show ONLY name and major (exclude _id)
    const query = { gpa: { $gt: 3.5 } };
    const options = {
        projection: { _id: 0, name: 1, major: 1 }
    };

    const cursor = students.find(query, options);

    // Convert cursor to array to view data
    const results = await cursor.toArray();
```

```
    console.log(results);
}
```

6.3 Update

Methods: `updateOne()` , `updateMany()` , `replaceOne()` **Update Operators:**

- `$set` : Modifies value of field(s).
- `$inc` : Increments a value.
- `$unset` : Removes a field.
- `$push` : Adds to an array.

Example:

```
async function updateStudentData(db) {
    const students = db.collection('students');

    // Scenario: Bob switched major to Physics and GPA increased by 0.1
    const filter = { name: "Bob" };
    const updateDoc = {
        $set: { major: "Physics" },
        $inc: { gpa: 0.1 }
    };

    const result = await students.updateOne(filter, updateDoc);
    console.log(`${result.matchedCount} document(s) matched, ${result.modifiedCou
}
```

6.4 Delete

Methods: `deleteOne()` , `deleteMany()`

Example:

```
async function cleanupData(db) {
    const students = db.collection('students');

    // Delete all students with GPA < 2.0
    const result = await students.deleteMany({ gpa: { $lt: 2.0 } });
    console.log(`Deleted ${result.deletedCount} students.`);
}
```

7. Unit Summary & Revision

7.1 Middleware Cheat Sheet

Syntax	Scope	Description

`app.use(fn)`	Global	Runs on every request.
`app.use('/path', fn)`	Path-specific	Runs only on requests starting with `/path`.
`app.get('/', fn, handler)`	Route-specific	Runs only for this specific route.
`(err, req, res, next) => {}`	Error	Triggered only when `next(err)` is called.

7.2 MongoDB Command Quick Reference

Operation	SQL Equivalent	MongoDB Command
Insert	`INSERT INTO`	`db.col.insertOne(doc)`
Select All	`SELECT *`	`db.col.find({})`
Select Specific	`SELECT col1`	`db.col.find({}, {projection: {col1: 1}})`
Where	`WHERE age > 18`	`db.col.find({age: {$gt: 18}})`
Update	`UPDATE table SET`	`db.col.updateOne(filter, {$set: ...})`
Delete	`DELETE FROM`	`db.col.deleteOne(filter)`

8. Exam Preparation

8.1 Common Mistakes & Exam Traps

1. **Middleware Ordering:**
   - *Mistake:* Placing static file middleware *after* your route handlers.
   - *Consequence:* The server tries to match a route first. If your route is `app.get('/:id')`, it might catch `style.css` as an ID.
   - *Fix:* Always place `app.use(express.static(...))` near the top.
2. **Async/Await in DB Ops:**
   - *Mistake:* Forgetting `await` on `find().toArray()` or `insertOne()`.
   - *Consequence:* The function returns a pending Promise, not the data. The exam question asking for output will be incorrect if you write "[Object object]" instead of "Promise { <pending> }".
3. **Destructive Updates:**
   - *Mistake:* Using `update` without operators like `$set`.
   - *Consequence:* In older MongoDB versions (or `replaceOne`), this replaces the *entire* document with just the update fields. Always check if the question implies "Patching"

(use `$set`) or "Replacing".

8.2 Practice Questions

Section A: Multiple Choice (1 Mark each)

1. **Which Express middleware serves static files?** a) `express.file()` b) `express.static()` c) `express.public()` d) `express.assets()` *Answer: b*
2. **In MongoDB, what does the projection** `{ name: 1, _id: 0 }` **do?** a) Returns all fields except name. b) Returns only name and the hidden ID. c) Returns only name and excludes the ID. d) Updates the name to 1. *Answer: c*
3. **Which function call correctly passes control to the next middleware?** a) `return true;` b) `next();` c) `continue();` d) `res.next();` *Answer: b*

Section B: Short Answer (5 Marks)

Q: Explain the difference between Application-level and Error-handling middleware in Express. *Model Answer:* Application-level middleware is bound to an instance of the `app` object using `app.use()` or `app.METHOD()` . It handles general request processing and typically takes 3 arguments `(req, res, next)` . Error-handling middleware is defined specifically to handle errors that occur during the request cycle. It is distinguished by its signature, which **must** have 4 arguments: `(err, req, res, next)` . Express recognizes the 4-argument signature and only invokes it when `next()` is called with an error parameter.

Section C: Long Answer / Coding (10 Marks)

Q: Write a Node.js script using Express and the native MongoDB driver that:

1. Connects to a database named "library".
2. Provides a POST route `/add-book` to insert a book (title, author).
3. Provides a GET route `/books` that returns all books but excludes the internal MongoDB ID.
4. Handles database connection errors gracefully.

Solution Hint:

```
const express = require('express');
const { MongoClient } = require('mongodb');
const app = express();
app.use(express.json()); // Middleware to parse JSON body

const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);
let db;

// Connection
async function startServer() {
    try {
        await client.connect();
        db = client.db('library');
        app.listen(3000, () => console.log('Server Ready'));
```

```
    } catch (e) {
        console.error("DB Error", e);
    }
}
startServer();

// Routes
app.post('/add-book', async (req, res) => {
    try {
        const result = await db.collection('books').insertOne(req.body);
        res.status(201).json({ message: 'Book added', id: result.insertedId });
    } catch (e) {
        res.status(500).send('Error inserting book');
    }
});

app.get('/books', async (req, res) => {
    try {
        const books = await db.collection('books')
            .find({}, { projection: { _id: 0 } }) // Select command equivalent
            .toArray();
        res.json(books);
    } catch (e) {
        res.status(500).send('Error fetching books');
    }
});
```

9. Case Study: The "BlogEngine" Mini-Project

Context: You are building a blog platform. **Requirement:**

1. Serve uploaded user avatars (Static Files).
2. Log every request for analytics (Middleware).
3. Store blog posts with varying tags and comment structures (NoSQL/MongoDB).

Architecture Analysis:

- **Static Serving:** You use `app.use('/uploads', express.static('uploads'))`. This allows users to access images via `site.com/uploads/user1.jpg`.
- **Database Choice:** A SQL database would require a rigid table for Posts, Comments, and Tags, requiring complex JOINs to render a single page. MongoDB allows storing comments *embedded* within the Post document.
    - *Document Structure:*

```
{
  "title": "My First Post",
  "content": "Hello World...",
  "tags": ["tech", "node", "web"],
  "comments": [
     { "user": "Dave", "text": "Great post!" },
     { "user": "Alice", "text": "Helpful." }
  ]
```

}